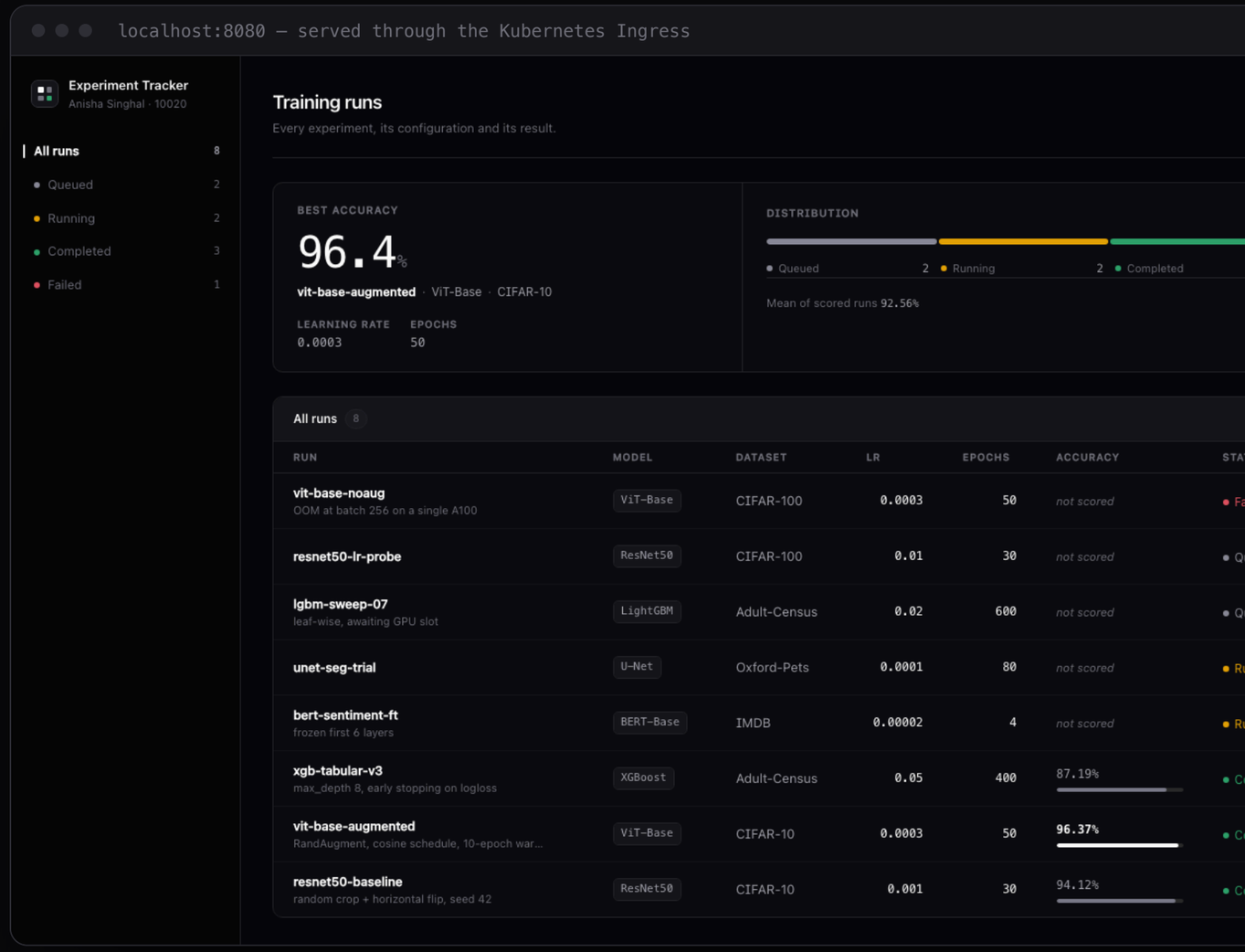
ML Experiment Tracker

A tracker for machine-learning training runs, taken from a laptop to a monitored, GitOps-managed Kubernetes cluster — with the infrastructure provisioned on AWS by Terraform.



Anisha Singhal
Enrollment 10020 · github.com/anisha-singhal/ml-experiment-tracker

Which configuration produced the good number?

One record per training run

Name, model, dataset, learning rate, epochs, status and accuracy — filterable by status, with totals, the best run and the mean.

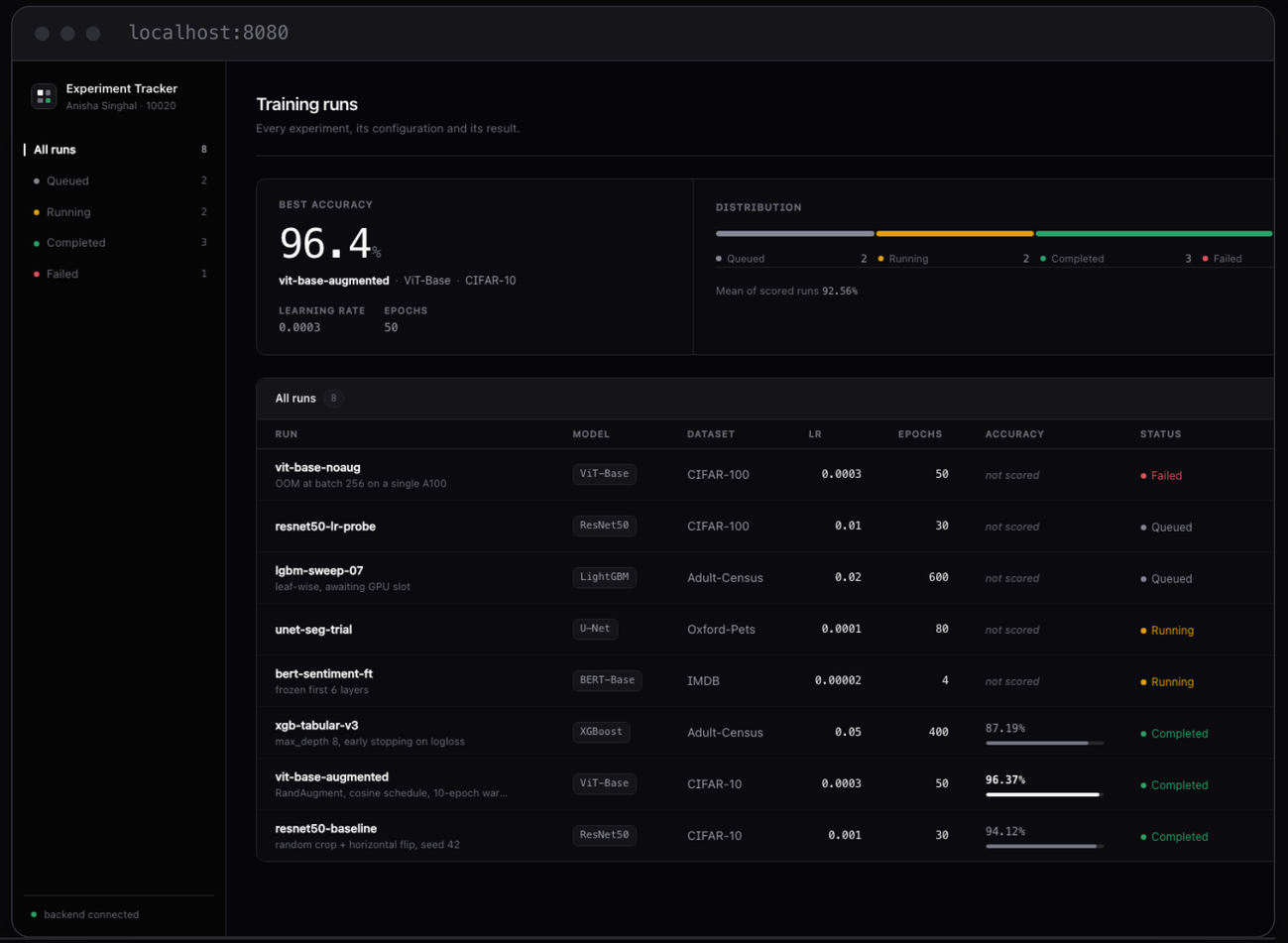
A status lifecycle

queued → running → completed, or failed. Each transition is an API call the dashboard makes.

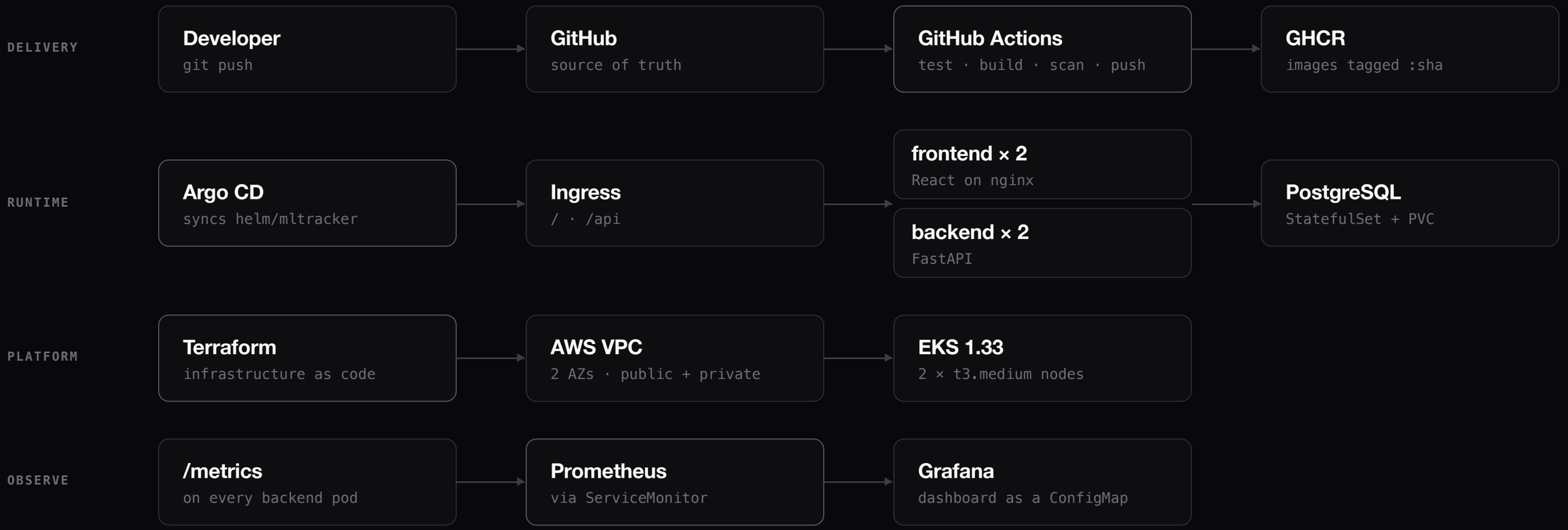
Accuracy is nullable, not zero

A queued run has no score yet, which is a different thing from having scored 0.0. The distinction holds in the schema, the API and the UI.

9	17	2×2	8
REST endpoints	pytest cases	pod replicas	Grafana panels



Four lanes, one chain



Local: kind, 3 nodes · Cloud: EKS in ap-southeast-2, created and destroyed by Terraform

Tech stack

APPLICATION

API	FastAPI 0.143
ORM and migrations	SQLAlchemy 2.1 · Alembic
Validation	Pydantic 2.14
Database	PostgreSQL 16
Frontend	React 18 · Vite 5
Tests	pytest · 17 cases

DELIVERY

CI/CD	GitHub Actions
Registry	GHCR · tagged by commit SHA
Security scanning	Trivy 0.75

PLATFORM

Containers	Docker · multi-stage · non-root
Orchestration	Kubernetes · kind and EKS 1.33
Packaging	Helm 3
Routing	ingress-nginx
GitOps	Argo CD 2.13
Infrastructure as code	Terraform 1.9 · AWS

OBSERVABILITY

Metrics	prometheus-fastapi-instrumentator
Collection	Prometheus · kube-prometheus-stack
Dashboards	Grafana

Every version pinned; every image rebuilt and rescanned on each push

Every push runs the whole gate

Test
pytest × 17

→

Build
frontend bundle

→

Image
docker, matrix

→

Scan
trivy

→

Gate
HIGH / CRITICAL

→

Push
ghcr.io · :sha

Scanned before it is pushed

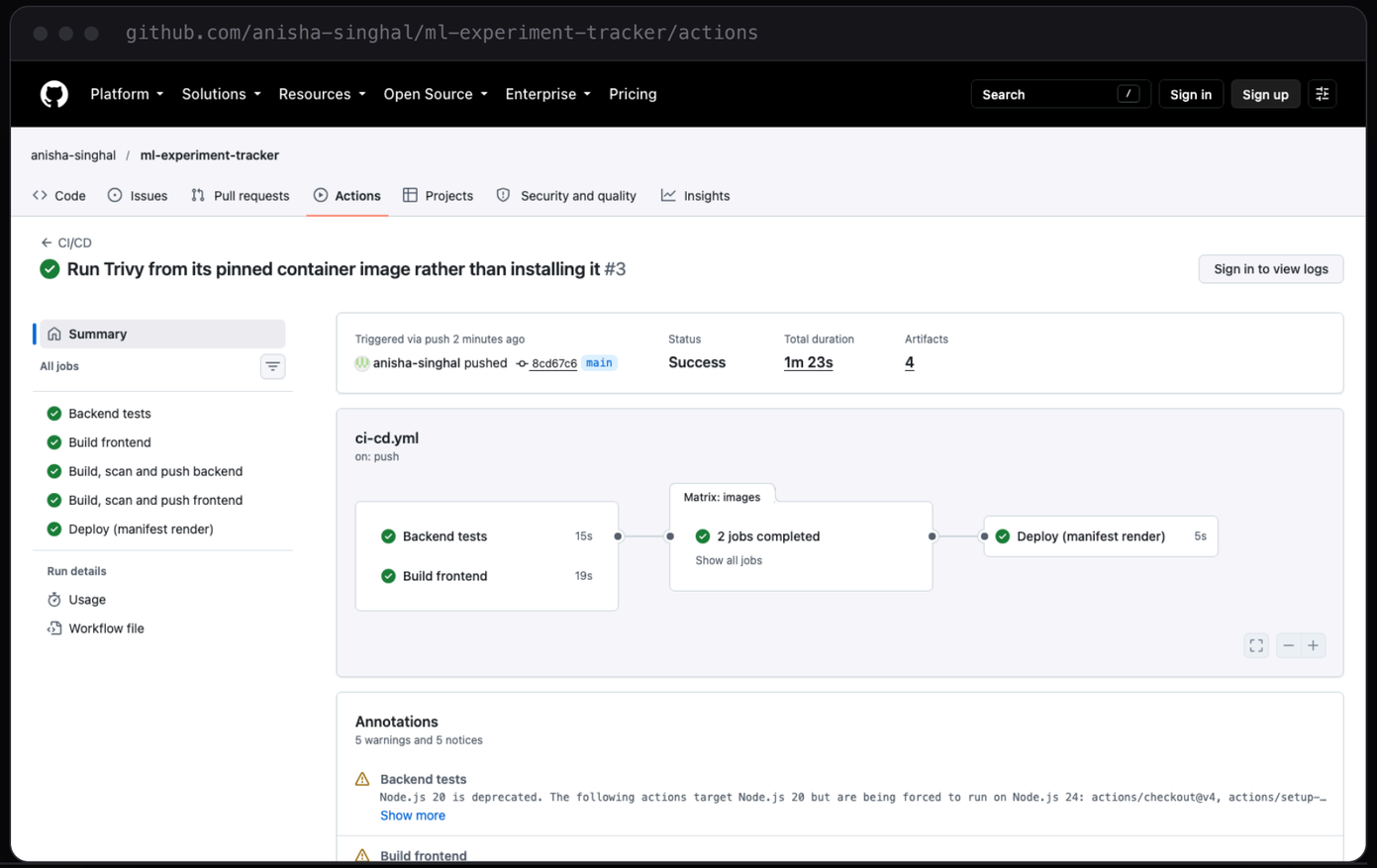
The image is loaded into the runner's Docker daemon, scanned there, and pushed only if the gate passes. A vulnerable image never reaches the registry.

Tagged with the commit SHA

Never :latest. Every running pod traces back to the exact commit that built it, and a rollback is a tag change, not a rebuild.

Fan-in, not a chain

Tests and the frontend build run in parallel; both must pass before any image is built.



The first scan failed both images. Good.

```
$ trivy image --severity HIGH,CRITICAL --ignore-unfixed --exit-code 1 <image>
```

BACKEND · FIXABLE HIGH

3 → 0

All three in **starlette 0.41.3** — not a direct dependency; FastAPI pins it. **CVE-2026-48818**: StaticFiles did not reject Windows UNC paths, so a crafted request could make the server open a share on an attacker's host. Fixed by upgrading FastAPI 0.115 → 0.143.

FRONTEND · FIXABLE HIGH + CRITICAL

41 → 0

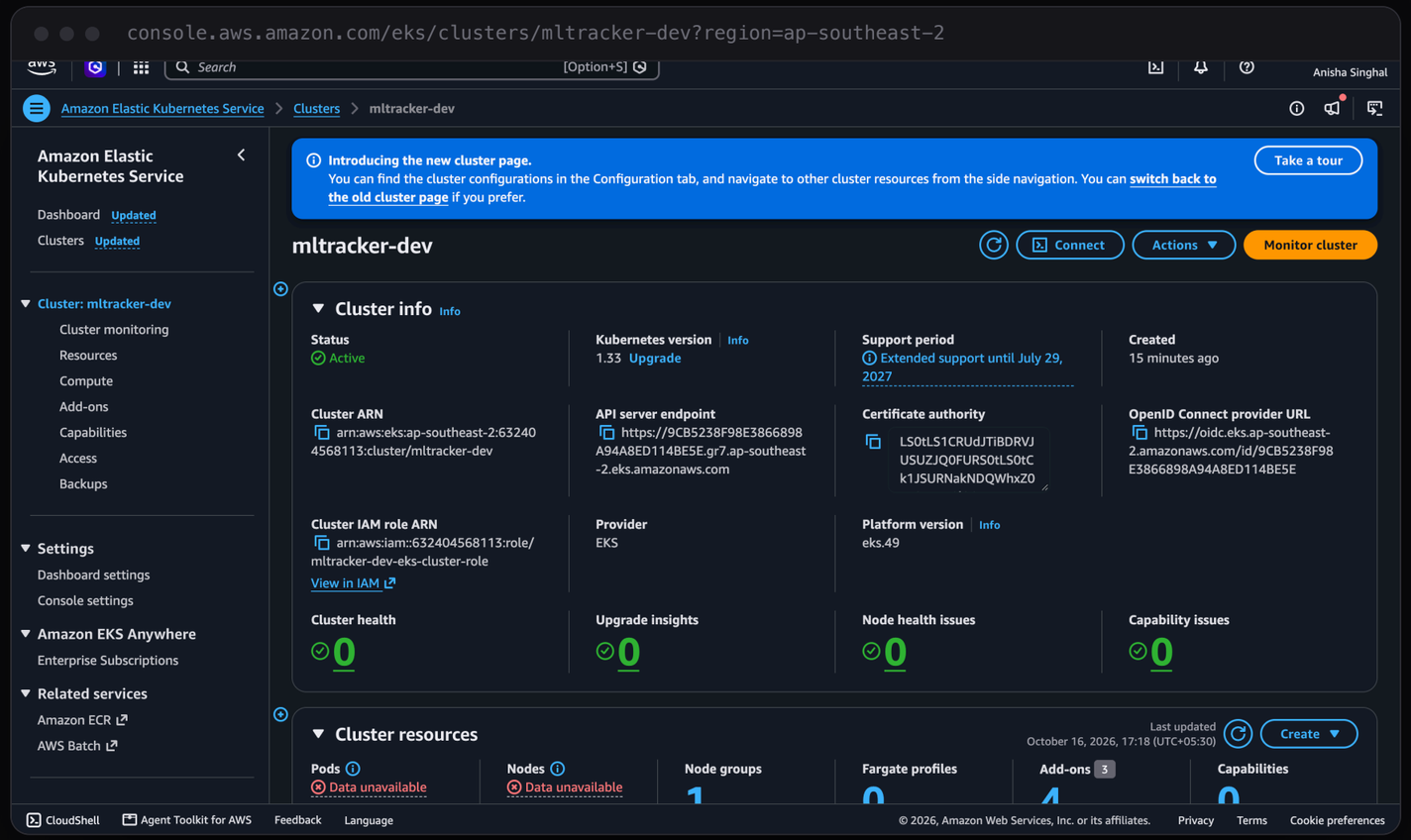
None of it this project's code — all inherited from the **nginx 1.27-alpine** base image, including 2 CRITICAL in libssl. A pinned base tag is rebuilt less often than Alpine ships fixes. Fixed with 1.29-alpine plus **apk upgrade** at build time.

WHY --IGNORE-UNFIXED

The backend still reports **44 HIGH with no upstream fix**. Failing every build on a CVE nobody has patched yet blocks delivery for a reason this repository cannot act on — so the gate fails only on what can be fixed, and the rest is tracked in docs/trivy-output.txt rather than hidden.

Terraform on real AWS

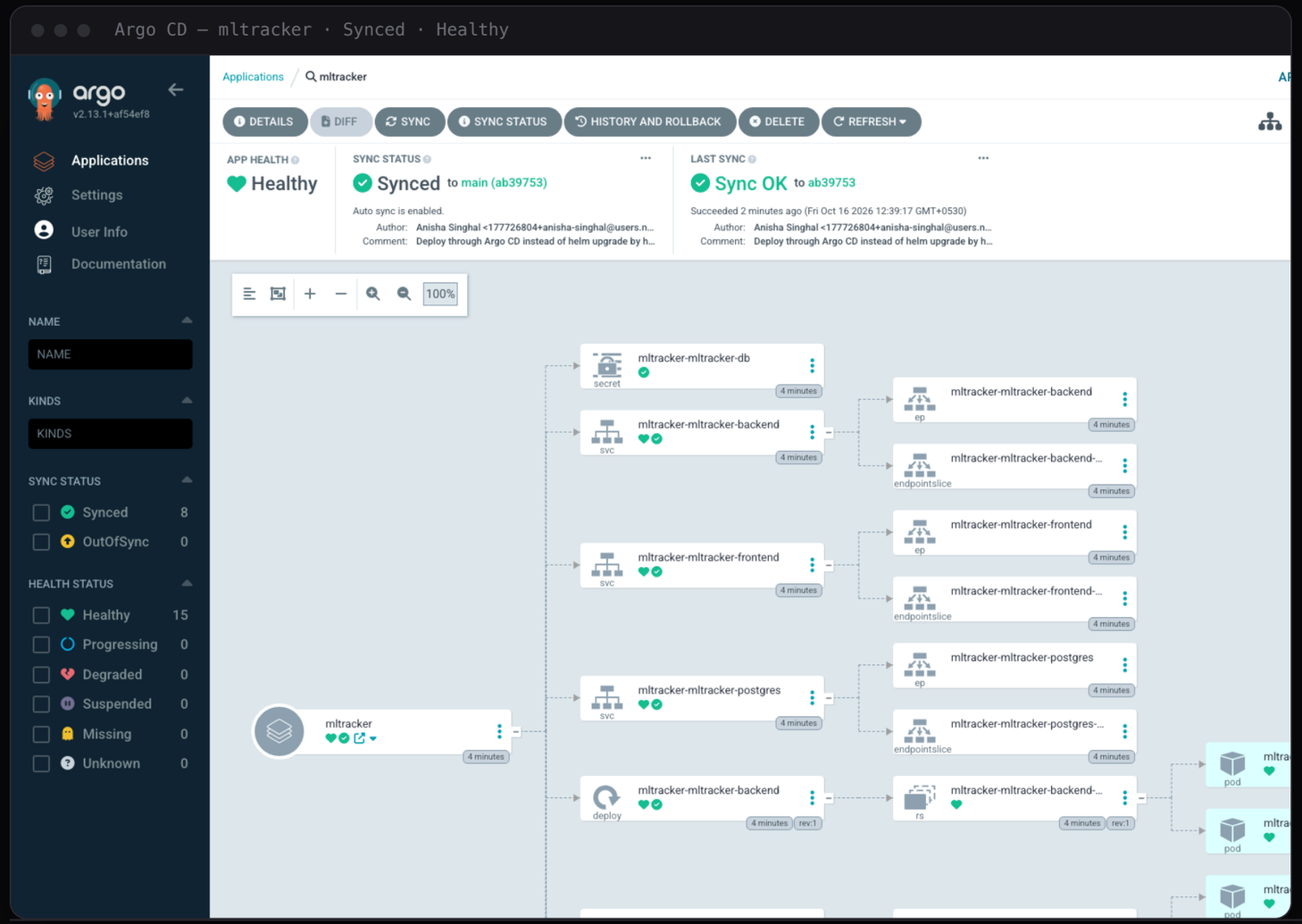
Region	ap-southeast-2
VPC	10.0.0.0/16
Subnets	2 public + 2 private · 2 AZs
Egress	IGW + 1 NAT gateway
EKS control plane	1.33 · Active
Node group	2 × t3.medium · Ready



WHAT COST AN HOUR

Every call returned **explicit deny in a service control policy** — the account is pinned to one region and I was calling Mumbai. The error named the service, never the region.

The deployment is a commit



Argo CD syncs the Helm chart

Nobody runs helm upgrade by hand. The synced revision is the Git commit it rendered.

Images pinned to a full SHA

Changing what runs means editing values-gitops.yaml and pushing.

Drift is reverted in five seconds

Scaling the frontend to 1 by hand put it back to 2 — the cluster cannot quietly diverge from Git.

Migrations run as initContainers

A Helm pre-install hook runs before the database exists, so it timed out every time.

Measured, not assumed

/metrics → ServiceMonitor → Prometheus

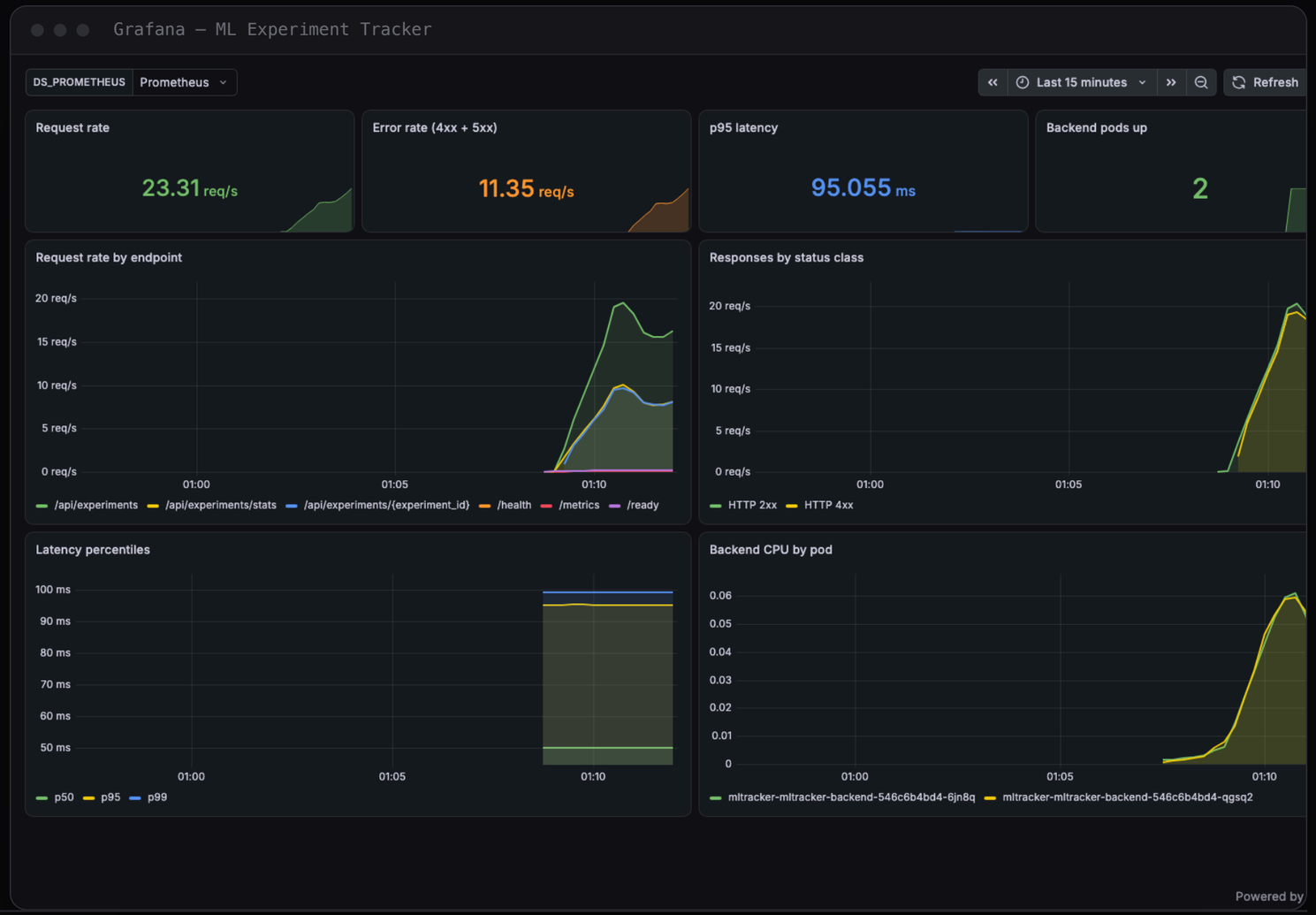
The scrape target is declared in the chart, so both backend pods are found as they come and go.

The dashboard is a ConfigMap

Version-controlled, not clicked together in the UI and lost on the next reinstall.

The load generator sends failures on purpose

404s and 422s mixed into the traffic. A dashboard that only ever shows 200s proves nothing about its error panels.



What actually broke

01

Ordering guarantees are the whole game

A Helm hook orders against the release; an initContainer orders against the pod. Only one of them runs after the database exists.

02

A security gate is only honest if it can fail

The first Trivy run broke both builds. Scanning after the push would have looked identical and protected nothing.

03

Transitive dependencies are where the CVEs live

Nothing here imports starlette or libssl. Both shipped inside the images anyway.

04

Read AccessDenied as "this request", not "this service"

A request carries a region, a principal and a resource. The message rarely says which one failed.

05

Measure before you fix

Twice I "fixed" layout bugs by eye. Both times, measuring in the browser showed the real cause was somewhere else.

Each one is written up in the README under "Things that broke, and why"

One chain, end to end

commit — test — build — scan — push :sha — Argo CD — Kubernetes — Ingress — Prometheus — Grafana

• 17 / 17	• 0	• Synced	• 2 / 2
tests passing in CI	fixable HIGH or CRITICAL CVEs	Argo CD, healthy, self-healing	EKS nodes Ready, then destroyed

Anisha Singhal

Enrollment 10020 · github.com/anisha-singhal/ml-experiment-tracker

Thank you